

lab.bitsmasher.net Manual

franklin

August 2026

Contents

1	Introduction	5
1.1	Purpose and Scope	5
2	Hardware Inventory	7
2.1	Desk Setup	7
2.2	Hardware Inventory	7
2.2.1	Desk Setup	7
2.2.2	Server and Compute Hosts	7
2.2.3	Historical / Decommissioned Hosts	8
2.2.4	Planned Hardware Expansion	8
3	Network Infrastructure	9
3.1	Addressing and Subnets	9
3.2	Network Infrastructure	9
3.2.1	Addressing Scheme	9
3.2.2	DHCP	9
3.2.3	DNS	9
3.2.4	Firewall and Access Control	10
3.2.5	Physical Topology	10
3.3	DNS	10
3.4	DNS Configuration	10
3.4.1	BIND Overview	10
3.4.2	Zones Maintained	10
3.4.3	DNS Server Location	10
3.4.4	Molecule Test Harness	11
3.4.5	DNS Failure Impact	11
4	Core Services	13
4.1	Network Time (NTP)	13
4.2	Network Time (NTP)	13
4.2.1	Overview	13
4.2.2	The Time Host	13
4.2.3	Configuration on Client Hosts	13
4.2.4	Installation and Setup Steps	14
4.2.5	Verification on Clients	14
4.2.6	Notes for Deployment	15
4.3	Kerberos Authentication	15
4.4	Kerberos Authentication	15
4.4.1	Realm Configuration	15
4.4.2	Access and Key Management	15
4.4.3	Principal Conventions	15
4.4.4	DNS/Kerberos Integration	15
4.4.5	Keytab Management	16
4.5	LDAP Directory Services	16
4.6	LDAP Directory Services	16
4.6.1	Server Configuration	16
4.6.2	Outage Details	16

4.6.3	Monitoring Approach	16
4.6.4	Recovery Steps	16
4.6.5	Directory Structure (Expected)	17
5	Security	19
5.1	Authentication and Access Control	19
5.2	Network Security	19
5.3	Certificate and TLS Management	19
5.4	Credential Management	19
5.5	Backup and Disaster Recovery	19
5.6	Container and Kubernetes Security	20
5.7	Automated Security Scanning	20
6	Storage	21
6.1	NFS Mounts and Backup	21
6.2	Storage Infrastructure	21
6.2.1	NFS Mounts	21
6.2.2	Snapshot and Backup Strategy	21
6.2.3	Local Storage Considerations	21
6.2.4	LFS Policy Reminder	22
6.3	LFS Policy	22
7	Internal PyPI Repository	23
7.1	Overview	23
7.2	Internal PyPI Repository	23
7.2.1	Architecture	23
7.2.2	Configuration via Ansible	23
7.2.3	Usage from Client Machines	23
7.2.4	Package Maintenance	24
8	Container Orchestration	25
8.1	k3s Cluster	25
8.2	Kubernetes (k3s) Cluster	25
8.2.1	Architecture	25
8.2.2	Version Pinning	25
8.2.3	Container Runtime	25
8.2.4	Cluster Topology	25
8.2.5	kubeconfig Access	26
9	Playbook Automation	27
9.1	Ansible Automation	27
9.1.1	Current Landscape (August 2026)	27
9.1.2	Molecule Testing – Status and Deprecation	27
10	lab/franklin Collection	29
10.1	Collection Manifest (galaxy.yml)	29
10.2	Role Inventory	29
10.3	Workspace Structure	30
10.4	Build System (Autotools)	30
11	Testing from Stargate	33
11.1	Molecule Test Harnesses on stargate.lab.bitsmasher.net	33
11.1.1	Test Environment	33
11.1.2	Test Execution Flow	33
11.1.3	Example: DNS Role Testing	33
11.1.4	Known Limitations	34
11.1.5	Running Tests Manually	34
11.1.6	Test Results Dashboard	34
11.1.7	pytest-ansible Transition Plan	34

11.1.8 Pending Tasks and Todos	34
12 Additional Infrastructure	35
12.1 Blowfish – Database Host	35
12.2 Blowfish – Database Host	35
12.2.1 Host Identity	35
12.2.2 Role in the Lab	35
12.2.3 Current Status	35
12.2.4 Planned Configuration (OpenBSD)	36
12.3 Minecraft Server	36
12.4 Minecraft Server	36
12.4.1 Host and Location	36
12.4.2 World and Chunk Storage	36
12.4.3 Dreamland Connection	36
12.4.4 Server Management via Ansible	36
13 Disaster Recovery	37
13.1 Disaster Recovery	37
13.1.1 Critical Dependencies Chain	37
13.1.2 Failure Scenarios and Recovery	37
13.1.3 Backup and Restoration	38
13.1.4 Prevention: Monitoring	38
14 Troubleshooting Guide	39
14.1 Troubleshooting Guide	39
14.1.1 SSH Connection Failures	39
14.1.2 Service-Specific Diagnostics	40
14.1.3 Common Ansible Troubleshooting	40
14.1.4 General Debugging Checklist	40
14.1.5 Quick Reference Commands	41
Infrastructure Services	43
14.2 Network Time (NTP)	43
14.2.1 Overview	43
14.2.2 The Time Host	43
14.2.3 Configuration on Client Hosts	43
14.2.4 Installation and Setup Steps	44
14.2.5 Verification on Clients	44
14.2.6 Notes for Deployment	44
14.3 Storage Infrastructure	45
14.3.1 NFS Mounts	45
14.3.2 Snapshot and Backup Strategy	45
14.3.3 Local Storage Considerations	45
14.3.4 LFS Policy Reminder	45
File Services: NFS	47
14.4 What We Know About NFS	47
14.5 Storage Infrastructure	47
14.5.1 NFS Mounts	47
14.5.2 Snapshot and Backup Strategy	47
14.5.3 Local Storage Considerations	47
14.5.4 LFS Policy Reminder	47
14.6 How to Test the NFS Ansible Role	48
14.7 Overview	48
14.8 Prerequisites	48
14.9 Running the Test	48
14.10What the Test Validates	48
14.11Current Test Limitations	48
14.12How to Fix the Test Harness	49

14.13	Integration Test Targets	49
14.14	Molecule Test Checklist for Future Work	49
Automation & Tooling		51
14.15	Ansible Automation	51
14.15.1	Current Landscape (August 2026)	51
14.15.2	Molecule Testing – Status and Deprecation	51
lab/franklin Collection		53
14.16	Collection Manifest (galaxy.yml)	53
14.17	Role Inventory	53
14.18	Workspace Structure	54
14.19	Build System (Autotools)	54
Testing from Stargate		57
14.20	Molecule Test Harnesses on stargate.lab.bitsmasher.net	57
14.20.1	Test Environment	57
14.20.2	Test Execution Flow	57
14.20.3	Example: DNS Role Testing	57
14.20.4	Known Limitations	58
14.20.5	Running Tests Manually	58
14.20.6	Test Results Dashboard	58
14.20.7	pytest-ansible Transition Plan	58
14.20.8	Pending Tasks and Todos	58
14.21	Ansible-Linting Infrastructure	58
14.21.1	Tools and Configuration	59
14.21.2	Coverage Status (44 Roles)	59
14.21.3	Proposed Linting Pipeline	59
14.21.4	Linting Exceptions by Role	59
Terraform Infrastructure Management		61
14.22	DigitalOcean Infrastructure	61
14.22.1	Provisioned Resources	61
14.22.2	Key DNS Records	61
14.22.3	Configuration Notes	61
14.23	Google Cloud Platform Infrastructure	61
14.23.1	Resources Managed	62
14.24	Security Scanning	62
14.25	Testing	62

Chapter 1

Introduction

1.1 Purpose and Scope

This manual documents the lab.bitmasher.net infrastructure – network, hardware, services, and procedures. It is intended as a reference for current operations and a foundation for onboarding new administrators.

Chapter 2

Hardware Inventory

2.1 Desk Setup

Franklin's desk hosts chonk, the primary gateway workstation. Sly's desk hosts a GAMING Windows PC at the main office near music.lan's monitor. Both have access to all lab infrastructure.

2.2 Hardware Inventory

2.2.1 Desk Setup

- **Franklin's desk:** chonk (10.10.8.60) – GAMING Windows PC, primary workstation, Gateway host running OpenClaw
- **Sly's desk:** GAMING Windows computer (Twitch: https://www.twitch.tv/sly_b0rg) – at main office near music.lan monitor
- Both sit at the main office where music.lan's monitor is displayed

2.2.2 Server and Compute Hosts

Core Infrastructure (Debian 12)

Host	IP	User	Role
chonk	10.10.8.60	franklin/openclaw	Gateway host, primary workstation
stargate	10.10.16.66	openclaw	Ansible collection workspace, build host
skynet	10.10.16.10	openclaw/franklin	Minecraft server, IP-direct login (hostname resolves oddly)
time	10.10.12.2	openclaw/franklin	Stratum-1 NTP server with GPS reference
wonderland	178.62.60.55	franklin/openclaw	Public cloud VM (Debian 12, 6.1 kernel)
music.lan	192.168.86.38	root	Desktop machine via skynet route

Jetson Cluster (~87 days uptime)

All three Jetsons run Ubuntu 18.04 with ed25519 key auth (openclaw permanent, franklin has password 123 as fallback):

Host	IP	Status
node900	10.10.12.90	Online, key auth
node901	10.10.12.91	Online, key auth
node903	10.10.12.93	Online, key auth
node902	10.10.12.92	Off / no route to host (likely powered down)

KDC and Directory Services

Host	IP	Status
odroid-c1 / kdc1	10.10.12.254	Online (242d uptime), root key auth, KDC for lab.bitsmasher.net Kerberos realm
ldap/bbb1	10.10.13.1	SSH reachable as root; slapd failed Dec 2025 (TLS cert issue)

2.2.3 Historical / Decommissioned Hosts

- **snowy**: Hard disk physically removed. /mnt/snowy is now just a mounted drive (not SSH-accessible). Should be cleaned from DNS and SSH config.
- **blowfish**: Previously at 10.10.12.15, reassigned to 10.10.14.85. Key not authorized yet – unreachable from chonk but port 22 is open. Expected role: database host with OpenBSD.
- **node902**: Powered off; no route to host from any known peer.

2.2.4 Planned Hardware Expansion

- blowfish as the primary database server (OpenBSD planned)
- head2 for cluster management (referenced in network_update.sh but unreachable)
- *k3s_serverandk3s_agentrolesforKubernetesacrossnodeinfrastructure*

Chapter 3

Network Infrastructure

3.1 Addressing and Subnets

The lab spans multiple private subnets organized by function, from the main office (10.10.8.0/21) to the backbone (10.10.16.0/) and specialized service zones.

3.2 Network Infrastructure

3.2.1 Addressing Scheme

The lab uses private IPv4 addressing across multiple subnets, organized by function:

Subnet	Allocation	Purpose
10.10.8.0/21	chonk, chonk-wifi, music.lan (via skynet)	Main office / desk area
10.10.12.0/	time, node90x, odroid-c1, blowfish, ns1	Core infrastructure
10.10.13.0/	ldap/bbb1	Directory services (moved to working hosts)
10.10.14.0/	blowfish (reassigned)	New allocation for blowfish
10.10.15.0/	femputer	Guest / temporary hosts
10.10.16.0/	stargate, skynet, edge-t	Lab backbone / staging

Note: The full subnet masks vary by role and DHCP scope configuration on the Dream Machine (DHCP server).

3.2.2 DHCP

The `dhcp` Ansible role manages the DHCP server configuration stored at `roles/dhcp/files/dhcpd.conf`. Key reserved hosts include:

- **chonk:** 10.10.8.60, MAC fc:9d:05:01:27:02 – Franklin’s desk
- **chonk-wifi:** 10.10.8.61 – wireless client
- **chonk-ten-gig-1:** 10.10.8.?? – dedicated ten-gig link
- **blowfish:** 10.10.8.15 / 10.10.12.15, MAC 98:b7:85:21:ad:77 – database host
- **femputer:** 10.10.15.1, MAC d8:bb:c1:af:21:17 – temporary/guest

3.2.3 DNS

The DNS infrastructure is managed by the `dns` Ansible role (BIND/named). Zones maintained include:

- Forward zone: db.home.lab
- Reverse zones for each subnet
- A records and PTR records for all lab hosts

DNS role molecule testing confirms BIND packages install correctly and `named.conf.options` are written. The DNS server (ns1) has been offline during recent audits, which explains why many hosts in the 10.10.x.0/24 range cannot resolve – ns1 is a single point of failure for lab DNS resolution.

3.2.4 Firewall and Access Control

- The Dream Machine provides built-in DHCP and handles basic routing between subnets
- odroid-c1 serves as the KDC and likely has pf firewall rules managing cross-subnet Kerberos traffic
- OpenBSD hosts (blowfish planned) use pf for host-level firewalls
- stargate has configured SSH access from chonk via openclaw key pair

3.2.5 Physical Topology

The lab spans multiple physical locations and subnets:

- Main office where Franklin and Sly have their desks monitors (music.lan)
- Server racks hosting core infrastructure (time, odroid-c1/KDC)
- Jetson cluster (node900–node903) for GPU compute
- Public cloud (wonderland/178.62.60.55) for external-facing services

Routing between the 192.168.86.x subnet (music) and the rest of the lab requires hopping through skynet due to the non-standard routing configuration.

3.3 DNS

BIND on ns1 provides authoritative DNS for the lab, with forward and reverse zones plus Kerberos SRV records. Currently offline – see Disaster Recovery for recovery procedures.

3.4 DNS Configuration

3.4.1 BIND Overview

The lab uses BIND (named) as its authoritative DNS server. The DNS infrastructure is managed by the `dns` Ansible role in the lab-franklin collection, located at:

```
1 ansible/collections/ansible_collections/lab/franklin/roles/dns/
2
```

3.4.2 Zones Maintained

- **Forward zone:** db.home.lab – maps hostnames to IPs for all lab infrastructure
- **Reverse zones:** PTR records for each subnet (10.10.x.0/24 ranges)
- **SRV records:** Kerberos service discovery (`_kerberos._tcp`, `_kdc.tcp`)

3.4.3 DNS Server Location

The DNS server (ns1) is the single authoritative nameserver for the lab. It has been unreachable during recent infrastructure audits, which explains why many hosts cannot resolve hostnames – ns1 *is* the DNS server and it's offline.

When ns1 is available, it resolves all internal hostnames. Key entries include:

- time.lab.bitsmasher.net → 10.10.12.2 (NTP server)
- odroid-c1.lab.bitsmasher.net → 10.10.12.254 (KDC)
- ldap.lab.bitsmasher.net → 10.10.13.1 (OpenLDAP)
- blowfish.lab.bitsmasher.net → 10.10.14.85 (reassigned from 10.10.12.15)

3.4.4 Molecule Test Harness

The DNS role has a molecule test harness that confirms:

1. **Converge:** BIND packages installed, /etc/bind directory created, named.conf.options written
2. **Test sequence:** dependency → syntax → create (podman container) → prepare → converge
3. **Limitation:** molecule test uses Ubuntu 20.04 container with hostname "server1"; the role has a hostname gate that targets the real DNS server's hostname

The DNS molecule test confirms correct package installation and configuration file placement but does not verify actual name resolution (no live DNS server in the container test environment).

3.4.5 DNS Failure Impact

When ns1 is offline:

- Lab hosts fall back to /etc/hosts entries (which are authoritative but not dynamic)
- Kerberos KDC discovery via SRV records fails – clients need manual kdc = entry in krb5.conf
- DHCP reservations on the Dream Machine remain functional (DHCP server is separate from DNS)

The DNS role's molecule test harness ensures that when the server comes back online, its configuration will install and write correctly.

Chapter 4

Core Services

4.1 Network Time (NTP)

A two-tier NTP architecture provides time synchronization from the Stratum-1 GPS-referenced time host to all lab clients.

4.2 Network Time (NTP)

4.2.1 Overview

Time synchronization is one of the most critical and least-visible aspects of network management. All lab services – Kerberos authentication, TLS certificate validation, log correlation, distributed databases, and filesystem consistency – depend on time accuracy. The lab uses a two-tier NTP architecture:

1. **Stratum 1 source:** GPS-referenced clock on the `time` host (10.10.12.2) provides the lab's authoritative time reference.
2. **Stratum 2+ clients:** All other hosts sync to `time.lab.bitasmasher.net`.

4.2.2 The Time Host

- **Hostname:** `time.lab.bitasmasher.net` (alias: `time`)
- **IP:** 10.10.12.2
- **User:** `openclaw`
- **OS:** Debian 12 bookworm
- **Role:** Stratum-1 NTP server / reference clock source for the lab

The time host has a GPS unit connected (serial/tty interface) providing UTC discipline. Its `ntpd` is configured via `ntpsec` to serve the lab subnet and use the GPS PPS signal as the primary time reference.

4.2.3 Configuration on Client Hosts

The NTP configuration is managed by Ansible via the `ntp` role in the `lab-franklin` collection. Here is what a typical client configuration looks like (as seen on `stargate`):

```
1 # /etc/ntpsec/ntp.conf -- managed by Ansible
2 driftfile /var/lib/ntp/ntp.drift
3 leapfile /usr/share/zoneinfo/leap-seconds.list
4
5 statsdir /var/log/ntpstats/
6
7 statistics loopstats peerstats clockstats
8 filegen loopstats file loopstats type day enable
```

```

9 filegen peerstats file peerstats type day enable
10 filegen clockstats file clockstats type day enable
11
12 server time.lab.bitasmasher.net
13 restrict 10.10.8.0/21
14 restrict -4 default

```

Key points:

- **driftfile:** Tracks oscillator drift between NTP restarts. Critical for stability on hosts that reboot frequently (Jetson devices, test VMs).
- **statistics:** Logs loop/peer/clock stats daily in `/var/log/ntpstats/`. Use these to verify sync health.
- **server:** Points to the lab's time host – never `pool.ntp.org` for infrastructure hosts that need deterministic time sources.
- **restrict:**
 - `10.10.8.0/21` grants query access to the main lab subnet (used by other internal clients or monitoring tools).
 - `-4 default` blocks all IPv4 queries not explicitly allowed, providing a deny-by-default posture.

4.2.4 Installation and Setup Steps

For a new host that needs NTP:

1. Install `ntpsec` packages:

```

1 apt update && apt install -y ntpsec
2

```

2. Configure `/etc/ntpsec/ntp.conf` (either manually or via Ansible `ntp` role):

- (a) Set `server time.lab.bitasmasher.net` as the upstream reference.
- (b) Set `timezone: timedatectl set-timezone America/Denver` (or appropriate TZ).
- (c) Create the log directory if it doesn't exist: `mkdir -p /var/log/ntpstats`.

3. Start and enable the service:

```

1 systemctl enable --now ntpsec.service
2 timedatectl # verify "System clock synchronized: yes"
3

```

4.2.5 Verification on Clients

Check synchronization status with these commands:

```

1 # Check if the system clock is synchronized
2 timedatectl | grep "System clock synchronized"
3
4 # Check NTP peer status (if ntpq is available)
5 ntpq -p
6
7 # View recent sync logs
8 journalctl -u ntpsec --since today
9

```


4.2.6 Notes for Deployment

- The Ansible `ntp` role handles all of the above automatically across lab hosts. It installs packages, copies configuration, sets timezone, and creates log directories.
- For Jetson devices (node900–node903), drift compensation is especially important – they boot frequently and have less stable oscillators than desktop/server hardware.
- The GPS unit on the time host should be verified periodically: confirm it’s locking to satellites, not using its internal oscillator as a holdover source. Check this by running `ntptime` on the time host itself.
- If the time host goes offline and all clients lose sync beyond their maximum correction threshold (typically 128 seconds), systems may need manual time correction before Kerberos/TLS will function again.

4.3 Kerberos Authentication

The KDC on odroid-c1 runs MIT Kerberos for the `lab.bitsmasher.net` realm, managing authentication for all services.

4.4 Kerberos Authentication

4.4.1 Realm Configuration

- **Realm name:** `lab.bitsmasher.net`
- **KDC host:** `odroid-c1.lab.bitsmasher.net` (10.10.12.254)
- **KDC software:** MIT Kerberos KDC running on odroid-c1
- **Uptime:** 242 days (stable, rarely rebooted)

4.4.2 Access and Key Management

The KDC only has SSH key auth configured for the `root` user. The `franklin` user has been rejected by the KDC’s SSH configuration – direct root access is required for any management tasks.

Key file: `/.ssh/id_ed25519_openc1aw` (the standard `openc1aw` key).

4.4.3 Principal Conventions

Kerberos principals in the `lab.bitsmasher.net` realm should follow these conventions:

- Service principals: `service/FQDN@lab.BITSMASHER.NET`
- User principals: `username@lab.BITSMASHER.NET`
- Host principals: `host/FQDN@lab.BITSMASHER.NET`

4.4.4 DNS/Kerberos Integration

Kerberos relies on DNS SRV records for KDC discovery:

```

1 _kerberos._tcp.lab.bitsmasher.net IN SRV 0 0 88 odroid-c1.lab.bitsmasher.net.
2 _kerberos._udp.lab.bitsmasher.net IN SRV 0 0 88 odroid-c1.lab.bitsmasher.net.
3 _kdc.tcp.lab.bitsmasher.net      IN SRV 0 0 88 odroid-c1.lab.bitsmasher.net.
4
```

These records should be present in the BIND DNS zones on the lab’s DNS server. If `ns1` is offline (as has been the case), Kerberos clients may fail to discover the KDC automatically and will need manual configuration via `kdc = odroid-c1.lab.bitsmasher.net` in `/etc/krb5.conf`.

4.4.5 Keytab Management

Keytabs for service principals should be:

- Generated on the KDC host (odroid-c1) using `kadmin.local` or `kadmin -q`
- Transferred securely to target hosts (via scp with SSH keys)
- Set to mode 600 and owned by the appropriate service user

The lab-franklin Ansible collection should have a role for managing keytabs across hosts, though this may need to be verified against current KDC state.

4.5 LDAP Directory Services

OpenLDAP (slapd) on bbb1 provides directory-based identity management. Service has been offline since December 2025 due to a TLS certificate issue.

4.6 LDAP Directory Services

4.6.1 Server Configuration

- **Hostname:** ldap.lab.bitsmasher.net (alias: bbb1)
- **IP:** 10.10.13.1
- **Software:** OpenLDAP (slapd)
- **Status:** SSH reachable as root, but slapd service has been **failed since December 2025**

4.6.2 Outage Details

The slapd failure manifests as:

```
1 TLS init def ctx failed (-1)
2
```

This indicates a TLS certificate configuration issue – likely an expired, missing, or misconfigured CA certificate. The server is listening on its network interface (SSH works for root), but the LDAP daemon itself has not recovered since December 2025.

4.6.3 Monitoring Approach

Franklin maintains `/home/franklin/status.sh` which checks:

- slapd service status on bbb1/ldap
- ldapsearch queries for franklin and sly DN lookups – both fail because the directory is down

The script provides a simple pass/fail indicator for LDAP health without requiring complex monitoring infrastructure.

4.6.4 Recovery Steps

To restore the LDAP server:

1. SSH to ldap.lab.bitsmasher.net as root (ed25519 key)
2. Check slapd logs: `journalctl -u slapd --since "2025-12-13"`
3. Verify TLS certificate chain in `/etc/ssl` or the configured cert path
4. Update or regenerate the CA-signed certificate if expired
5. Restart slapd: `systemctl restart slapd`
6. Verify with ldapsearch for franklin and sly DNs

4.6.5 Directory Structure (Expected)

The OpenLDAP directory should contain entries for:

- User entries for franklin, Sly (slyborg), and other lab members
- Group/organizational unit entries for lab access control
- Service accounts for LDAP-authorized services (Kerberos integration, web auth)

Once restored, the LDAP directory serves as the central authentication source for the lab, complementing Kerberos for identity management.

Chapter 5

Security

5.1 Authentication and Access Control

SSH key management, password policy, and user accounts define access to the lab infrastructure. All automation hosts share a common ed25519 key pair for the openclaw user. The primary admin (franklin) has sudo NOPASSWD on skynet; root is restricted to KDC, ldap, and music host only.

5.2 Network Security

Subnet segmentation provides implicit isolation across the lab. The “prereq” Ansible role handles firewall exceptions for K3s API ports on each host (6443, 2379–2381). Both UFW (Debian) and FirewallD (RedHat-family) are supported.

5.3 Certificate and TLS Management

The “tls” role manages certificate provisioning across the lab. The OpenLDAP server experienced a TLS failure in December 2025 due to an expired certificate – this remains the primary ongoing TLS vulnerability.

5.4 Credential Management

Credentials use pass (password manager) with GPG encryption. The Stash House project manages encrypted credential backups synced via bare git repos. Sensitive Ansible variables are stored in ansible-vault files that should never be committed to version control.

5.5 Backup and Disaster Recovery

NFS-mounted clusterfs2 provides shared storage with redundant copies across hosts. DNS zones are version-controlled in BIND configuration files.

Service availability:

- **NTP stratum-1** (time.lab.bitmasher.net) – Online
- **Kerberos KDC** (odroid-c1.lab.bitmasher.net) – Online, 242 days uptime
- **DNS BIND** (ns1.lab.bitmasher.net) – Offline since December 2025
- **OpenLDAP** (ldap.lab.bitmasher.net) – Offline since December 2025
- **Minecraft** (skynet.lab.bitmasher.net) – Online

Disaster recovery priority: NTP > DNS > Kerberos KDC > LDAP. Time sync failure cascades to all Kerberos authentication; DNS failure makes hosts unreachable by name.

5.6 Container and Kubernetes Security

The k3s cluster is hardened with version pinning (2.1.x), Containerd runtime with NVIDIA GPU support, TLS for all API communication (k3s default), and pod security policies via Kubernetes native admission controllers.

5.7 Automated Security Scanning

The `.github` directory contains these automated workflows: `bandit.yml` (Bandit Python security), `tfsec-sarif.yml` (Terraform IaC security), `trivy.yml` + `trivy-cb.yml` (container/filesystem vulnerability scanning), `bash_check.yml` (GNU autotools build validation with “make security”), and `markdown.yml` (markdownlint). All run on every pull request and main branch push.

Chapter 6

Storage

6.1 NFS Mounts and Backup

clusterfs2 provides shared NFS storage across hosts. Credential backups use GPG-encrypted pass entries synced via bare git repos (Stash House project).

6.2 Storage Infrastructure

6.2.1 NFS Mounts

The lab relies on NFS for shared storage across hosts. Key mount points:

- **/mnt/clusterfs2**: Primary cluster workspace mounted on chonk and stargate (from the NFS server). This is where Ansible roles, molecule tests, and project files live.
- **/mnt/snowy**: Formerly a full NFS share; now just a mounted drive after the hard disk was physically removed. Should be audited for stale mount entries.

The NFS role in lab-franklin's Ansible collection handles:

- Server-side export configuration (/etc/exports)
- Client-side fstab management and mount points
- Mount path conventions per role (previously /mnt/clusterfs, /mnt/backup1, /mnt/storage1,2,3)

Note: The NFS role has hostname gates ('snowy' in inventory_hostname) that prevent testing on molecule containers. Hardcoded mount paths depend on real NFS servers being online. This is a known limitation of the test harness.

6.2.2 Snapshot and Backup Strategy

The lab uses bare git repos for version control and backup:

- GPG-encrypted repositories synced across hosts for credential protection (Stash House project)
- Pass-based password store with GPG backend
- No LFS – ABSOLUTE RULE: do not use git-lfs anywhere. Microsoft once held Minecraft chunk files hostage when a repo hit the 10GB GitHub limit with LFS enabled.

6.2.3 Local Storage Considerations

- Jetson nodes (node90x): limited eMMC storage; rely on NFS for workspace data
- chonk: large local disk, primary build and gateway host
- stargate: Debian 12 box with full Ansible workspace mirror

6.2.4 LFS Policy Reminder

ABSOLUTE RULE: Do NOT use git-lfs anywhere in any lab-franklin repository.

This applies across all repos: training-ai, minecraft, Ansible collections, everything. The incident with Minecraft chunk files being held hostage by GitHub when the repo hit 10GB with LFS is a real precedent – never risk it.

6.3 LFS Policy

Absolute rule: no git-lfs in any lab repository – see the storage section for details on the incident that made this policy necessary.

Chapter 7

Internal PyPI Repository

7.1 Overview

An internal PyPI server provides Python packages to development machines without depending on external PyPI.org servers. Packages are stored on NFS-mounted storage and served via HTTP from the web document root.

7.2 Internal PyPI Repository

The lab runs an internal PyPI package repository to provide Python packages to development machines without depending on external PyPI servers. This avoids network bottlenecks, supports offline development, and enables use of proprietary or lab-specific packages not available on PyPI.org.

7.2.1 Architecture

The PyPI directory lives at `/mnt/storage1/LAB/pypi`. A symlink is created from the web server's document root (`/var/www/html/pypi`) pointing to this directory, making packages accessible via HTTP.

Access URL pattern:

```
1 http://<storage-host>/pypi/simple/  
2
```

This follows the PEP 503 simple repository API structure expected by pip.

7.2.2 Configuration via Ansible

The `pypi_internal` role in the `lab/franklin` collection automates this setup:

- **Role path:** `roles/pypi_internal/`
- **Variables:**
 - `html_dir`: web root (`/var/www/html`)
 - `pypi_dir`: package directory (`/mnt/storage1/LAB/pypi`)
- **Task:** Creates a symbolic link from `$html_dir/pypi` → `$pypi_dir`
- **Playbook integration:** Included in the `cluster/storage` playbook alongside NFS, LDAP, Kerberos, DNS, and `ntp` roles

7.2.3 Usage from Client Machines

Configure pip to use the internal PyPI server:

```
1 # In ~/.pip/pip.conf or /etc/pip.conf:
2 [global]
3 index-url = http://<storage-host>/pypi/simple/
4
5 # Optionally trust host if not using HTTPS
6 trusted-host = <storage-host>
7
```

Or use pip's `indexurl` flag directly:

```
1 pip install --index-url http://<storage-host>/pypi/simple/ some-package
2
```

7.2.4 Package Maintenance

To add packages to the internal PyPI, place them in the pypi directory with PEP 503-compliant directory structure:

- Package directories (lowercase names)
- File links inside each package directory pointing to the actual wheel/sdist files
- The web server must have read access to `/mnt/storage1/LAB/pypi`

Packages can be built and added using standard pip tools:

```
1 pip install --no-index --find-links=/mnt/storage1/LAB/pypi/simple/ <package>
2
```

Chapter 8

Container Orchestration

8.1 k3s Cluster

k3s provides Kubernetes across the infrastructure with version pinning (2.1.x), containerd runtime, and NVIDIA Container Toolkit for GPU workloads.

8.2 Kubernetes (k3s) Cluster

8.2.1 Architecture

The lab uses k3s (Lightweight Kubernetes from Rancher) for container orchestration across the infrastructure:

- **k3s_server**: Master node running k3s control plane (host: head2, referenced but unreachable in current audit)
- **k3s_agent**: Worker nodes that register with the server and receive workloads
- **GPU nodes**: Specialized workers for NVIDIA container runtime and compute workloads

8.2.2 Version Pinning

k3s version is pinned to the 2.1.x series (specifically 2.1.5 as of the latest `network_update.sh`). This prevents surprise breakage from upstream k3s releases. The pinning policy follows:

```
1 K3S_VERSION="2.1.5"
2 curl -sfL https://get.k3s.io | sh -s - \
3     --write-kubeconfig-mode 644
4
```

8.2.3 Container Runtime

k3s uses containerd as the default runtime, with NVIDIA Container Toolkit for GPU workloads:

- **nvidia-container-toolkit**: Installed on GPU nodes via the `network_update.sh` automation
- **containerd config**: Located at `/var/lib/rancher/k3s/agent/etc/containerd/config.toml`
- NVIDIA runtime configured in containerd to expose GPU devices to containers

8.2.4 Cluster Topology

Node Role	Target Hosts	Notes
k3s_server	head2 (planned)	Control plane; not yet online
k3s_agent	node900, node901, node903	Jetson nodes as workers
nvidia_nodes	GPU-equipped hosts	Container Toolkit + k3s agent

8.2.5 kubeconfig Access

After installation, kubeconfig is written to `/etc/rancher/k3s/k3s.yaml` with mode 644. Cluster access requires:

- SSH key auth to the `k3s_server` host
- Reading the kubeconfig file (or copying it securely)
- `kubectl --kubeconfig /etc/rancher/k3s/k3s.yaml cluster-info` to verify connectivity

The `k3s_server` role and `k3s_agent` role in `lab-franklin` handle the installation and configuration of these nodes via Ansible.

Chapter 9

Playbook Automation

9.1 Ansible Automation

9.1.1 Current Landscape (August 2026)

As of August 2026, the Ansible ecosystem is split between two distribution paths:

- **Ansible community package** – the traditional monolithic install (pip install ansible). Current version is 13.x (built on ansible-core 2.20). This includes 85+ collections with thousands of modules and plugins bundled together. Follows semantic versioning, releases two major versions per year plus monthly minor releases.
- **ansible-core** – the minimal core engine. Maintains three versions at a time (current + two older). Versioned independently from the community package (2.19, 2.20, 2.21). Releases every four weeks for minor updates.

EOL versions as of August 2026: Ansible 10–12 and ansible-core 2.15–2.18 are EOL. Do not use these in new work. Only Ansible 13.x / ansible-core 2.20 is current; 14.0.0/2.21 is in development.

9.1.2 Molecule Testing – Status and Deprecation

Molecule (the canonical integration testing framework for Ansible roles and playbooks) remains **active** as of August 2026 but its role has shifted:

- Molecule v26.4.x is the latest series, released June 2026. It still supports only the latest two major versions of Ansible (N/N-1).
- Molecule is being superseded as the preferred test strategy by:
 - **pytest-ansible** – pytest extension for testing Ansible module/plugin Python code directly.
 - **tox-ansible** – integration with tox to run tests across multiple Python interpreters and ansible-core versions simultaneously.
 - Execution Environments (containers) as the primary test target, tested via **ansible-navigator**.
- The **community.molecule** collection still exists but new projects should prefer pytest-ansible + tox-ansible for role/test harness work. Molecule is not yet officially deprecated – it is maintained on a best-effort basis while the community transitions.
- For existing molecule test harnesses (such as those in lab-franklin’s Ansible collection at [ansible/collections/ansible-test](#)) they will continue to work with ansible-core 2.20 and pytest-6.x but should be audited for:
 - Podman driver compatibility (molecule continues using podman or docker as default driver)
 - Python version requirements (molecule requires Python ≥ 3.9)
 - Plugin deprecations in newer collections

Note: For lab-franklin’s molecule tests, keep testing with podman for now. The container-based approach is still valid; only the *preferred tooling around it* is shifting toward pytest-ansible + execution environments.

Chapter 10

lab/franklin Collection

The lab/franklin Ansible collection lives at:

```
1 /mnt/clusterfs2/workspace/lab-franklin/ansible/collections/ansible_collections/lab/
2 franklin/
```

It is the central automation artifact for the entire lab.bitasmasher.net infrastructure, managing provisioning, configuration, and testing across all hosts.

10.1 Collection Manifest (galaxy.yml)

The collection follows standard Ansible Galaxy conventions with namespace **lab** and name **franklin**:

- **Version:** 1.0.0
- **License:** MIT
- **Author:** franklin <franklin@bitasmasher.net>
- **Repository:** <https://github.com/devsecfranklin/lab-franklin>
- **Documentation:** <https://github.com/devsecfranklin/lab-franklin/docs>

10.2 Role Inventory

The collection contains roles organized under the `roles/` directory. As of August 2026, the following roles are maintained with molecule test harnesses:

```
1 roles/
2 apt_mirror/           # APT mirror configuration
3 beagleboard/          # BeagleBoard device provisioning
4 common/               # Shared base configuration (bootstrap.sh, common utils)
5 container_registry/   # Docker/container registry setup
6 ctfd/                 # CTFd platform deployment
7 desktop/              # Desktop environment provisioning
8 docker/               # Docker engine installation
9 documentation/        # Documentation generation roles
10 dhcp/                 # DHCP server configuration (dhcpd.conf)
11 dns/                  # BIND/named DNS server
12 golang/               # Go language toolchain
13 jetson-nano/          # NVIDIA Jetson Nano provisioning
14 k3s_agent/            # k3s worker node
15 k3s_server/           # k3s control plane server
16 k8s/                  # Kubernetes generic setup
17 latex/                # LaTeX environment
18 logging/              # Centralized logging
19 music/                # Music service configuration
```

```

20 nfs/                # NFS server/client
21 nix/                # Nix package manager
22 openbsd/           # OpenBSD host bootstrap (blowfish target)
23     tasks/main.yml  # Includes files.yml, timezone setup
24     files/          # bootstrap.sh, login.conf, hosts entries
25 paloalto/          # Palo Alto firewall config
26 prereq/            # Prerequisite packages
27 pypi_internal/      # Internal PyPI repository setup
28     tasks/main.yml  # Symlink pypi_dir -> html_dir/pypi
29     vars/main.yml   # html_dir=/var/www/html, pypi_dir=/mnt/storage1/LAB/pypi
30 python/            # Python environment
31 raspberrypi/        # Raspberry Pi provisioning
32 samba/             # Samba file sharing
33 security/          # Hardening and security baselines
34 shell/             # Shell configuration
35 ssh/               # SSH server hardening
36 tls/               # TLS certificate management
37 chonk/             # chonk-specific configuration
38 cluster/           # Cluster-wide settings
39

```

Roles currently **missing molecule tests**: edge, mail, odroid, website (incomplete roles). These have tasks/main.yml but no test harness.

10.3 Workspace Structure

The full project layout:

```

1 /workspace/lab-franklin/
2 Makefile.am          # Top-level autotools target (Python, Docker, dev)
3 configure.ac         # Autotools config (autoconf)
4 bootstrap.sh         # Cross-platform bootstrapper
5 network_update.sh    # One-shot maintenance script (hardened 2026-08-02)
6 ansible/             # ANSIBLE_HOME root
7     playbook.yml     # Main playbook
8     hosts            # Inventory file
9     collections/      # Collections path
10         ansible_collections/ # Galaxy namespace
11             lab/franklin/    # The lab/franklin collection
12                 galaxy.yml
13                 roles/      # All role modules (37 total)
14                 docs/latex/ # LaTeX docs build (autotools)
15 container/           # Docker/Podman configs
16 terraform/           # Terraform infrastructure definitions
17 bin/                 # Shared utility scripts (common.sh)
18 docs/manual/         # Lab manual documentation (this document)
19

```

10.4 Build System (Autotools)

The project uses GNU Autotools for the root-level build:

- **configure.ac**: Defines package (lab-franklin v0.2), libtool support, Python ≥ 3.9 requirement, podman/ansible/gpg checks, git version tagging
- **Makefile.am**: Top-level targets include test (ansible-lint + molecule), security (venv bootstrap), dev (Python venv in BUILDDIR)
- **bootstrap.sh**: Cross-platform bootstrapper that detects OS (Debian/RedHat/OpenBSD/macOS/Linux) and installs platform-specific packages, then runs `aclocal` $\rightarrow$ `autoreconf` $\rightarrow$ `automake` $\rightarrow$ `configure`

- **docs/manual/Makefile.am:** Standalone autotools stub for LaTeX docs build (clean target removes *.aux files)

The `network_update.sh` script was hardened on 2026-08-02 with: `set -euo pipefail`, auto-resolved paths, pinned K3s version validation, `ANSIBLE_HOMEEenvironmentchecks,removeddangerousclush/apt-getblastradius, anddeadcodere moval.Itrunsthemainplaybookatansible/playbook.ymlasitsprimaryaction.`

Chapter 11

Testing from Stargate

11.1 Molecule Test Harnesses on stargate.lab.bitsmasher.net

The stargate host (10.10.16.66, Debian 12 bookworm) serves as the primary testing platform for the lab/franklin Ansible collection. Testing is run from `/mnt/clusterfs2/workspace/lab-franklin/ansible/collections`

11.1.1 Test Environment

Molecule tests on stargate use:

- **Driver:** podman (containers, not Docker – avoids daemon dependency)
- **Base image:** Ubuntu 20.04 for most roles, with role-specific variations
- **Execution user:** openclaw user (has sudo -n, no password required)
- **Python:** ≥ 3.9 (required by both ansible and molecule)

11.1.2 Test Execution Flow

For a given role, the molecule test cycle is:

1. **dependency:** Install collection dependencies from `galaxy.yml`
2. **syntax:** Validate playbook YAML syntax
3. **create:** Launch podman container (e.g., "server1") as the test target host
4. **prepare:** Verify role prerequisites within the container
5. **converge:** Run ansible-playbook against the container – this is where the role's tasks are executed
6. **test/verify:** Optional pytest sequence runs inside the container to validate convergence

11.1.3 Example: DNS Role Testing

The DNS role molecule test confirmed:

- Podman creates an Ubuntu 20.04 container (hostname "server1")
- Bind9 packages install correctly
- `/etc/bind` directory is created with `named.conf.options` written
- Molecule platform name was changed from "molecule-ubuntu" to "server1" to match the role's hostname gate
- Group assignment updated to "dns_servers" for proper role targeting

11.1.4 Known Limitations

Some roles cannot be fully tested on molecule containers:

- **nfs**: Hostname gates ('snowy' in `inventory_hostname`) and hardcoded mount paths depend on real NFS servers
- **openbsd**: The role targets OpenBSD hosts specifically; molecule tests run on Ubuntu containers by default
- **jetson-nano**: Hardware-specific (NVIDIA Jetson); molecule tests only verify file placement, not actual provisioning

11.1.5 Running Tests Manually

To test a specific role from the stargate host:

```
1 cd /mnt/clusterfs2/workspace/lab-franklin/ansible/collections/ansible_collections/
  lab/franklin/roles/<role>/molecule
2 molecule converge --scenario-name default
3
```

To verify convergence results:

```
1 molecule test --scenario-name default
2
```

If the podman driver is unavailable, fall back to docker:

```
1 molecule converge --driver-name docker
2
```

11.1.6 Test Results Dashboard

Testing output logs are captured in the molecule scenario directory under `logs/`. The full lab-franklin test suite (all roles with molecule tests) is run via the top-level Makefile.am `test` target:

```
1 make test
2 # Runs ansible-lint --all-roles and molecule converge per role
3
```

11.1.7 pytest-ansible Transition Plan

As molecule is being superseded, new tests should use pytest-ansible where possible:

- Migrate the test directory structure from `molecule/` to a `tests/` directory with pytest configuration
- Use ansible-test for collection sanity tests (built into ansible-core)
- Keep molecule only where full environment isolation is needed (e.g., DNS zone file validation, NTP time sync testing)

11.1.8 Pending Tasks and Todos

- **TODO: Re-establish internal PyPI server on stargate.** The `pypi_internal` role is ready (role exists, molecule test added Aug 2026) but the actual PyPI service needs to be provisioned on the storage host. Requires: NFS mount of `/mnt/storage1` available on stargate, web server (nginx/apache) installed and configured, pip packages populated into `/mnt/storage1/LAB/pypi` with PEP 503 structure. Run: `ansible-playbook ... -l storage --tags pypi_internal` once the infrastructure is in place.

Chapter 12

Additional Infrastructure

12.1 Blowfish – Database Host

blowfish is the planned database host (OpenBSD). Currently unreachable; SSH key not yet authorized.

12.2 Blowfish – Database Host

12.2.1 Host Identity

- **Hostname:** blowfish.lab.bitsmasher.net (alias: blowfish)
- **IP:** 10.10.12.15
- **MAC:** 98:b7:85:21:ad:77 (from DHCP reservation)
- **OS:** OpenBSD (planned; current DNS/DHCP entry references it as an OpenBSD host)

12.2.2 Role in the Lab

Blowfish is provisioned as a **database host** within the lab infrastructure. It was intended to serve as a dedicated database server, separate from the primary infrastructure hosts (chonk, stargate, skynet).

The OpenBSD role in the lab-franklin Ansible collection includes configuration references specific to blowfish:

- Login configuration uses `blowfish` password hashing algorithm (`localcipher=blowfish,a`) in `/etc/login.conf`.
- The bootstrap.sh for OpenBSD hosts includes a direct `/etc/hosts` entry for blowfish (10.10.12.15).
- DHCP reservation is configured in the lab's `dhcpd.conf` role.

12.2.3 Current Status

Blowfish has been **unreachable** from chonk since the initial infrastructure audit. The IP 10.10.12.15 resolved from DNS (stargate's `/etc/hosts`), but authentication was denied – no matching SSH key was authorized on that host.

Key references:

- `network_update.sh`: blowfish is the default target for the `openbsd_bootstrap()` function.
- OpenBSD role tasks: includes commented-out ansible setup command for blowfish (indicating the host was expected to be online).

The host may need network investigation – it resolved from DNS, had a DHCP reservation, and MAC address on file, suggesting it was once part of the lab but has since gone offline or been decommissioned.

12.2.4 Planned Configuration (OpenBSD)

When brought online, the expected setup for blowfish is:

- SSH key-based authentication via `/.ssh/authorized_keys`
- OpenBSD's built-in `pf` firewall with custom rules
- Database software stack (likely PostgreSQL or MariaDB on OpenBSD)
- Ansible-managed configuration via the `lab-franklin` collection, using the `openbsd` role as the base profile.

12.3 Minecraft Server

The Minecraft server runs on `skynet`, with world data stored on NFS and version-controlled via bare git repos.

12.4 Minecraft Server

12.4.1 Host and Location

The Minecraft server runs on **skynet.lab.bitsmasher.net** (10.10.16.10), one of the core lab infrastructure hosts alongside `stargate` and `chonk`. The Minecraft role is managed by the `minecraft` Ansible role in the `lab-franklin` collection.

Skynet's dual-role setup:

- Primary: Minecraft game server host
- Secondary: General-purpose infrastructure node (Ansible workspace, CI testing)

Note: `skynet` uses IP-based SSH login (10.10.16.10) rather than hostname – the hostname resolves oddly in SSH config due to a double-domain suffix issue.

12.4.2 World and Chunk Storage

Minecraft world data is stored on NFS-mounted storage (`clusterfs2`). The world files are version-controlled where practical, with the following constraint:

NO GIT-LFS: Chunk files and world data must not use `git-lfs`. The Minecraft repository hit GitHub's 10GB LFS limit in a prior incident where chunk files were held hostage. Use bare git repos for any versioned world data.

12.4.3 Dreamland Connection

The *Dreamland Manual* referenced in the `docs/manual` directory covers the Dreamland Minecraft server – this manual documents that specific server's configuration, players, and game rules. The physical infrastructure (`skynet` host) is shared between both operations.

12.4.4 Server Management via Ansible

The `minecraft` role handles:

- Minecraft server installation and version updates
- World backup procedures (NFS snapshots or bare git repos)
- Player whitelist management
- Mod/plugin configuration if applicable

Molecule testing for the `minecraft` role confirms correct file placement and configuration generation, though live server connectivity is not verified by the test harness.

Chapter 13

Disaster Recovery

13.1 Disaster Recovery

13.1.1 Critical Dependencies Chain

The lab infrastructure has a cascading dependency chain. A failure in one layer propagates:

1. **Time (NTP)** → if time drift exceeds 128 seconds, Kerberos tickets expire and TLS handshakes fail
2. **DNS (ns1/BIND)** → without DNS resolution, all hostname-based services (Kerberos SRV records, NTP upstream, LDAP) break
3. **Kerberos (odroid-c1/KDC)** → without the KDC, authentication fails for all Kerberos-authenticated services
4. **LDAP (bbb1/slapd)** → without LDAP, directory lookups fail and user/service account information is unavailable

Recovery priority should follow this chain in reverse: fix time first, then DNS, then KDC, then LDAP.

13.1.2 Failure Scenarios and Recovery

NTP Failure – All Hosts Lose Time Sync

Symptoms: `timedatectl` shows "System clock synchronized: no", Kerberos tickets rejected.

Recovery:

1. Verify GPS unit is locked on odroid-c1 (time host)
2. Check ntpd/ntpsec service: `systemctl status ntpsec`
3. If GPS is offline, enable holdover mode (internal oscillator continues at reduced accuracy)
4. On affected clients, force resync: `ntpdate -s time.lab.bitasmasher.net` or wait for ntpd's step-sync to kick in
5. Verify with `ntpq -p` on clients and `ntptime` on the server

DNS (ns1) Offline – Cascade Failure

Symptoms: hostnames don't resolve, Kerberos SRV discovery fails.

Recovery:

1. Check ns1 reachability: ping from any known-good host
2. If ns1 is up but not answering: check BIND (named) service status
3. If BIND has crashed due to config error: restore from last known-good named.conf
4. As a workaround, add manual entries to `/etc/hosts` on affected hosts until DNS recovers
5. Update `krb5.conf` with explicit `kdc = odroid-c1.lab.bitasmasher.net`

LDAP (bbb1) Down – No Directory Service

Symptoms: slapd failed, status.sh reports both slapd and ldapsearch as failing.

Recovery:

1. SSH to bbb1/lab.bitsmasher.net as root
2. Check TLS certificate validity in the configured cert path
3. Regenerate or replace expired certificates
4. Restart slapd: `systemctl restart slapd`
5. Verify with ldapsearch for franklin and sly DNs

KDC (odroid-c1) Offline – No Authentication

Symptoms: kinit fails for all realms, service authentication denied.

Recovery:

1. SSH to odroid-c1 as root (franklin user's key is not authorized)
2. Check kdc process: `kadmind/krb5kdc status`
3. Verify KDC database integrity: `kdb5_utllist`
4. Restart kerberos services if needed
5. Regenerate host keytabs for affected services via `kadmin.local`

13.1.3 Backup and Restoration

- Ansible playbooks (lab-franklin collection) are the single source of truth – they can reconstruct any configured state
- Bare git repos with GPG-encrypted pass entries handle credential backup
- KDC database (`$KRB5_KDB_FILE`) should be backed up regularly on odroid-c1
- DNS zone files should have local copies (in the DNS role's files/ directory)

13.1.4 Prevention: Monitoring

The current monitoring approach is manual via status.sh. For improved resilience, consider:

- Automated ping/SSH checks on critical hosts in HEARTBEAT.md
- Nagios/Zabbix-style monitoring on a dedicated host
- Email/slack alerts for service down conditions

The lab-franklin Ansible collection should eventually include a monitoring role to automate this.

Chapter 14

Troubleshooting Guide

14.1 Troubleshooting Guide

14.1.1 SSH Connection Failures

Host Key Mismatch

Symptom: @@@@ WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED! @@@@

Fix:

```
1 ssh-keygen -R <hostname>
2
```

This occurs when a host was re-imaged with the same IP. Check `/etc/ssh/ssh_config.d/` for stale entries.

Auth Denied (Port 22 Open, Key Rejected)

Symptom: TCP port 22 is reachable but SSH returns "Permission denied".

Common causes:

- No matching key in the remote host's `authorized_keys`
- The key file permissions are too open on the target (must be mode 600 for `.pub` and 700 for `/.ssh`)
- Wrong user context (e.g., trying `franklin@` when only `root` is configured)

Fix: Add the openclaw ed25519 key to the target host's `authorized_keys` file. The pubkey is:

```
1 ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIJb9mV02PpxD8VpzYCnu7192dTWMnGWUc3qh55BIeE1N
  openclaw@chonk
2
```

No Route to Host

Symptom: SSH hangs or returns "No route to host". The host is likely powered off. Common in Jetson nodes (node902, etc.) that are manually managed.

GSSAPI Interference

Symptom: SSH times out waiting for GSSAPI authentication before falling back to key auth.

Fix: Add to `/.ssh/config`:

```
1 Host *
2     GSSAPIAuthentication no
3     IdentitiesOnly yes
4     IdentityFile ~/.ssh/id_ed25519_openclaw
5
```

14.1.2 Service-Specific Diagnostics

Kerberos Authentication Failure

Symptom	Likely Cause
kinit fails with "clock skew"	NTP not synced – check time.host first
kinit fails with "KDC has no support for encryption type"	Wrong krb5.conf realm or KDC unavailable
kadmin.local works but kadmin (remote) fails	Port 749 firewall rule on odroid-c1
Ticket expired after reboot	Time drift during host downtime
Verify: <code>klist</code> , <code>kinit username@LAB.BITSMASHER.NET</code> , <code>ktutil</code> for keytabs.	

NTP Issues

Symptom	Likely Cause
ntpd not running	ntpsec service not enabled
"no server suitable for synchronization"	No upstream reachable (time host offline)
Clock gradually drifting	Missing driftfile or Jetson oscillator drift
Time jumps by several seconds	Large correction applied; step-sync mode engaged
Verify: <code>ntpq -p</code> , <code>timedatectl</code> , check <code>/var/log/ntpstats/</code> .	

NFS Mount Issues

Symptom	Likely Cause
mount hangs	NFS server unreachable or export doesn't list client
"Stale file handle"	Server-side data was deleted/recreated
Permission denied on mount	<code>/etc/exports</code> restricts the client IP
Verify: <code>showmount -e <server></code> , check NFS server's exports, verify client IP matches the export rule.	

14.1.3 Common Ansible Troubleshooting

Module Not Found / Missing Collection

```

1 ansible-galaxy collection install lab.franklin
2 # or for individual modules:
3 ansible-galaxy collection install community.general
4

```

Molecule Test Failures on New Hosts

If molecule tests fail on a new host, the most common causes are:

1. Podman not installed or permissions wrong (openclaw user needs podman group)
2. Docker driver configured instead of podman – edit `molecule.yml` to set driver
3. Python version mismatch (molecule requires ≥ 3.9)
4. Role hostname gates preventing test execution (e.g., the DNS role's hostname check for the real server vs. molecule container name)

14.1.4 General Debugging Checklist

When a service is down, work through this checklist:

1. Can you ping the host? ($\rightarrow$ network up?)
2. Is SSH responding? ($\rightarrow$ host OS running?)
3. Does the service process exist? (`systemctl status` or `ps aux`)

4. Are logs showing errors? (`journalctl -u <service> --since today`)
5. Can you connect to the service port from another lab host? (`nc -zv <host> <port>`)
6. Is DNS resolving correctly? (`dig` or `getent hosts`)
7. Is time synchronized? (`timedatectl`)

14.1.5 Quick Reference Commands

```
1 # Check if a host is reachable from chonk
2 ping -c 1 <hostname>
3 ssh -o ConnectTimeout=5 openclaw@<host> "echo alive"
4
5 # Check service status remotely
6 ssh openclaw@<host> "systemctl status <service>"
7
8 # Verify time sync across hosts
9 for host in stargate skynet chonk; do
10     ssh openclaw@$host "timedatectl | grep synchronized"
11 done
12
13 # Verify DNS resolution from a client
14 dig @time.lab.bitsmasher.net <hostname>
15
16 # Check Kerberos auth
17 kinit username@LAB.BITSMASHER.NET
18 klist
19
```


Infrastructure Services

14.2 Network Time (NTP)

14.2.1 Overview

Time synchronization is one of the most critical and least-visible aspects of network management. All lab services – Kerberos authentication, TLS certificate validation, log correlation, distributed databases, and filesystem consistency – depend on time accuracy. The lab uses a two-tier NTP architecture:

1. **Stratum 1 source:** GPS-referenced clock on the `time` host (10.10.12.2) provides the lab's authoritative time reference.
2. **Stratum 2+ clients:** All other hosts sync to `time.lab.bitsmasher.net`.

14.2.2 The Time Host

- **Hostname:** `time.lab.bitsmasher.net` (alias: `time`)
- **IP:** 10.10.12.2
- **User:** `openclaw`
- **OS:** Debian 12 bookworm
- **Role:** Stratum-1 NTP server / reference clock source for the lab

The time host has a GPS unit connected (serial/tty interface) providing UTC discipline. Its `ntpd` is configured via `ntpsec` to serve the lab subnet and use the GPS PPS signal as the primary time reference.

14.2.3 Configuration on Client Hosts

The NTP configuration is managed by Ansible via the `ntp` role in the `lab-franklin` collection. Here is what a typical client configuration looks like (as seen on `stargate`):

```
1 # /etc/ntpsec/ntp.conf -- managed by Ansible
2 driftfile /var/lib/ntp/ntp.drift
3 leapfile /usr/share/zoneinfo/leap-seconds.list
4
5 statsdir /var/log/ntpstats/
6
7 statistics loopstats peerstats clockstats
8 filegen loopstats file loopstats type day enable
9 filegen peerstats file peerstats type day enable
10 filegen clockstats file clockstats type day enable
11
12 server time.lab.bitsmasher.net
13 restrict 10.10.8.0/21
14 restrict -4 default
```

Key points:

- **driftfile:** Tracks oscillator drift between NTP restarts. Critical for stability on hosts that reboot frequently (Jetson devices, test VMs).

- **statistics:** Logs loop/peer/clock stats daily in `/var/log/ntpstats/`. Use these to verify sync health.
- **server:** Points to the lab's time host – never `pool.ntp.org` for infrastructure hosts that need deterministic time sources.
- **restrict:**
 - `10.10.8.0/21` grants query access to the main lab subnet (used by other internal clients or monitoring tools).
 - `-4 default` blocks all IPv4 queries not explicitly allowed, providing a deny-by-default posture.

14.2.4 Installation and Setup Steps

For a new host that needs NTP:

1. Install ntpsec packages:

```
1 apt update && apt install -y ntpsec
2
```

2. Configure `/etc/ntpsec/ntp.conf` (either manually or via Ansible `ntp` role):

- (a) Set `server time.lab.bitasmasher.net` as the upstream reference.
- (b) Set `timezone: timedatectl set-timezone America/Denver` (or appropriate TZ).
- (c) Create the log directory if it doesn't exist: `mkdir -p /var/log/ntpstats`.

3. Start and enable the service:

```
1 systemctl enable --now ntpsec.service
2 timedatectl # verify "System clock synchronized: yes"
3
```

14.2.5 Verification on Clients

Check synchronization status with these commands:

```
1 # Check if the system clock is synchronized
2 timedatectl | grep "System clock synchronized"
3
4 # Check NTP peer status (if ntpq is available)
5 ntpq -p
6
7 # View recent sync logs
8 journalctl -u ntpsec --since today
9
```

14.2.6 Notes for Deployment

- The Ansible `ntp` role handles all of the above automatically across lab hosts. It installs packages, copies configuration, sets `timezone`, and creates log directories.
- For Jetson devices (node900–node903), drift compensation is especially important – they boot frequently and have less stable oscillators than desktop/server hardware.
- The GPS unit on the time host should be verified periodically: confirm it's locking to satellites, not using its internal oscillator as a holdover source. Check this by running `ntptime` on the time host itself.
- If the time host goes offline and all clients lose sync beyond their maximum correction threshold (typically 128 seconds), systems may need manual time correction before Kerberos/TLS will function again.

14.3 Storage Infrastructure

14.3.1 NFS Mounts

The lab relies on NFS for shared storage across hosts. Key mount points:

- **/mnt/clusterfs2**: Primary cluster workspace mounted on chonk and stargate (from the NFS server). This is where Ansible roles, molecule tests, and project files live.
- **/mnt/snowy**: Formerly a full NFS share; now just a mounted drive after the hard disk was physically removed. Should be audited for stale mount entries.

The NFS role in lab-franklin's Ansible collection handles:

- Server-side export configuration (/etc/exports)
- Client-side fstab management and mount points
- Mount path conventions per role (previously /mnt/clusterfs, /mnt/backup1, /mnt/storage1,2,3)

Note: The NFS role has hostname gates ('snowy' in `inventory_hostname`) that prevent testing on molecule containers. Hardcoded mount paths depend on real NFS servers being online. This is a known limitation of the test harness.

14.3.2 Snapshot and Backup Strategy

The lab uses bare git repos for version control and backup:

- GPG-encrypted repositories synced across hosts for credential protection (Stash House project)
- Pass-based password store with GPG backend
- No LFS – ABSOLUTE RULE: do not use git-lfs anywhere. Microsoft once held Minecraft chunk files hostage when a repo hit the 10GB GitHub limit with LFS enabled.

14.3.3 Local Storage Considerations

- Jetson nodes (node90x): limited eMMC storage; rely on NFS for workspace data
- chonk: large local disk, primary build and gateway host
- stargate: Debian 12 box with full Ansible workspace mirror

14.3.4 LFS Policy Reminder

ABSOLUTE RULE: Do NOT use git-lfs anywhere in any lab-franklin repository.

This applies across all repos: training-ai, minecraft, Ansible collections, everything. The incident with Minecraft chunk files being held hostage by GitHub when the repo hit 10GB with LFS is a real precedent – never risk it.

File Services: NFS

14.4 What We Know About NFS

14.5 Storage Infrastructure

14.5.1 NFS Mounts

The lab relies on NFS for shared storage across hosts. Key mount points:

- **/mnt/clusterfs2**: Primary cluster workspace mounted on chonk and stargate (from the NFS server). This is where Ansible roles, molecule tests, and project files live.
- **/mnt/snowy**: Formerly a full NFS share; now just a mounted drive after the hard disk was physically removed. Should be audited for stale mount entries.

The NFS role in lab-franklin's Ansible collection handles:

- Server-side export configuration (/etc/exports)
- Client-side fstab management and mount points
- Mount path conventions per role (previously /mnt/clusterfs, /mnt/backup1, /mnt/storage1,2,3)

Note: The NFS role has hostname gates ('snowy' in `inventory_hostname`) that prevent testing on molecule containers. Hardcoded mount paths depend on real NFS servers being online. This is a known limitation of the test harness.

14.5.2 Snapshot and Backup Strategy

The lab uses bare git repos for version control and backup:

- GPG-encrypted repositories synced across hosts for credential protection (Stash House project)
- Pass-based password store with GPG backend
- No LFS – ABSOLUTE RULE: do not use git-lfs anywhere. Microsoft once held Minecraft chunk files hostage when a repo hit the 10GB GitHub limit with LFS enabled.

14.5.3 Local Storage Considerations

- Jetson nodes (node90x): limited eMMC storage; rely on NFS for workspace data
- chonk: large local disk, primary build and gateway host
- stargate: Debian 12 box with full Ansible workspace mirror

14.5.4 LFS Policy Reminder

ABSOLUTE RULE: Do NOT use git-lfs anywhere in any lab-franklin repository.

This applies across all repos: training-ai, minecraft, Ansible collections, everything. The incident with Minecraft chunk files being held hostage by GitHub when the repo hit 10GB with LFS is a real precedent – never risk it.

14.6 How to Test the NFS Ansible Role

14.7 Overview

The NFS role at `ansible/collections/ansible_collections/lab/franklin/roles/nfs` has a molecule test harness, but testing is complex due to hostname-dependent task gates. This chapter documents how to run tests and what to expect.

14.8 Prerequisites

- podman available on the test host (chonk tested successfully)
- The lab.franklin collection with all dependencies installed
- A molecule environment with an Ubuntu-based container (the role expects Debian/Ubuntu apt packages)

14.9 Running the Test

From the role directory:

```
cd /mnt/clusterfs2/workspace/lab-franklin/ansible/collections/ansible_collections/lab/franklin/roles
molecule converge --scenario-name default
molecule verify --scenario-name default
```

14.10 What the Test Validates

- **Package installation:** `nfs-kernel-server`, `nfs-common`, `rpcbind`, `nfs-client` are installed
- **Configuration file deployment:** `/etc/default/nfs-kernel-server`, `/etc/default/nfs-common`, `/etc/idmapd.conf` are written with correct content
- **Directory creation:** Export mount points (`/mnt/clusterfs`, `/mnt/storage{1,2,3}`) are created
- **Export template rendering:** Jinja2 templates generate valid `/etc/exports` content

14.11 Current Test Limitations

The role has two categories of gates that block full testing:

1. Server-side hostname gates (in `tasks/main.yml`)

```
when: 'snowy' in inventory_hostname
when: 'thelio' in inventory_hostname
```

Molecule containers cannot satisfy these conditions, so `nfs_server_*.yml` task files never execute. The server-side converge phase is effectively skipped.

2. molecule environment variable guards

Most mount and service-start tasks check:

```
when: HOMELAB_MOLECULE_TEST is not defined
```

This prevents actual mounts from happening during testing (which would require real NFS servers), but it also means the `nfs_client.yml` tasks that matter most for client-side validation never run their key stages.

14.12 How to Fix the Test Harness

To make molecule tests meaningful, the following changes are needed:

1. **Add test-specific inventory:** In `molecule/default/inventory.yml`, define a hostgroup (e.g., `nfs_servers`) that includes container hostnames matching the role's hostname gates. Example:

```
all:
  children:
    nfs_servers:
      hosts:
        server1:
          ansible_connection: docker
    nfs_clients:
      hosts:
        client1:
          ansible_connection: docker
```

2. **Add override variables in `molecule__prepare.yml`:** Set `nfs_exports_snowy` and `nfs_exports_thelio` to container-safe values that use local paths, so export template rendering can be validated without real storage.
3. **Create a test scenario for client-only testing:** A second molecule scenario with only `nfs_clients` inventory that tests mount directory creation, fstab line insertion, and `idmapd.conf` deployment. Use `state: present` instead of `state: mounted` in the molecule overrides so it doesn't try to connect to a live NFS server.
4. **Replace hostname gates with role variables:** Replace 'snowy' in `inventory_hostname` with a variable like `nfs_server_mode | default(false)` so tests can enable server tasks without depending on real hostnames.
5. **Add idempotence tests:** A second converge pass to verify that running the role again produces no changes (the `force: no` on `idmapd.conf` copy already helps here).

14.13 Integration Test Targets

An integration test target exists at `tests/integration/targets/nfs-server/` but it is minimal. For thorough testing, add:

- Export verification (check `showmount -e`)
- Mount/unmount cycle tests on client container
- Kerberos security validation (try mounting with and without `sec=krb5i`)
- Permission checks after mount (verify `root_squash` behavior)

14.14 Molecule Test Checklist for Future Work

Before declaring the NFS role test-ready:

Server-side converge passes (with hostname override in molecule inventory)

Client-side converge passes (mount directory creation, fstab entries)

Verify phase checks exports file content and configuration files

Idempotence test: second converge shows zero changed tasks

Integration test validates live mount on container

Automation & Tooling

14.15 Ansible Automation

14.15.1 Current Landscape (August 2026)

As of August 2026, the Ansible ecosystem is split between two distribution paths:

- **Ansible community package** – the traditional monolithic install (pip install ansible). Current version is 13.x (built on ansible-core 2.20). This includes 85+ collections with thousands of modules and plugins bundled together. Follows semantic versioning, releases two major versions per year plus monthly minor releases.
- **ansible-core** – the minimal core engine. Maintains three versions at a time (current + two older). Versioned independently from the community package (2.19, 2.20, 2.21). Releases every four weeks for minor updates.

EOL versions as of August 2026: Ansible 10–12 and ansible-core 2.15–2.18 are EOL. Do not use these in new work. Only Ansible 13.x / ansible-core 2.20 is current; 14.0.0/2.21 is in development.

14.15.2 Molecule Testing – Status and Deprecation

Molecule (the canonical integration testing framework for Ansible roles and playbooks) remains **active** as of August 2026 but its role has shifted:

- Molecule v26.4.x is the latest series, released June 2026. It still supports only the latest two major versions of Ansible (N/N-1).
- Molecule is being superseded as the preferred test strategy by:
 - **pytest-ansible** – pytest extension for testing Ansible module/plugin Python code directly.
 - **tox-ansible** – integration with tox to run tests across multiple Python interpreters and ansible-core versions simultaneously.
 - Execution Environments (containers) as the primary test target, tested via **ansible-navigator**.
- The **community.molecule** collection still exists but new projects should prefer pytest-ansible + tox-ansible for role/test harness work. Molecule is not yet officially deprecated – it is maintained on a best-effort basis while the community transitions.
- For existing molecule test harnesses (such as those in lab-franklin’s Ansible collection at **ansible/collections/ansible-test**) they will continue to work with ansible-core 2.20 and pytest-6.x but should be audited for:
 - Podman driver compatibility (molecule continues using podman or docker as default driver)
 - Python version requirements (molecule requires Python ≥ 3.9)
 - Plugin deprecations in newer collections

Note: For lab-franklin’s molecule tests, keep testing with podman for now. The container-based approach is still valid; only the *preferred tooling around it* is shifting toward pytest-ansible + execution environments.

lab/franklin Collection

The lab/franklin Ansible collection lives at:

```
1 /mnt/clusterfs2/workspace/lab-franklin/ansible/collections/ansible_collections/lab/
  franklin/
2
```

It is the central automation artifact for the entire lab.bitsmasher.net infrastructure, managing provisioning, configuration, and testing across all hosts.

14.16 Collection Manifest (galaxy.yml)

The collection follows standard Ansible Galaxy conventions with namespace **lab** and name **franklin**:

- **Version:** 1.0.0
- **License:** MIT
- **Author:** franklin <franklin@bitsmasher.net>
- **Repository:** <https://github.com/devsecfranklin/lab-franklin>
- **Documentation:** <https://github.com/devsecfranklin/lab-franklin/docs>

14.17 Role Inventory

The collection contains roles organized under the `roles/` directory. As of August 2026, the following roles are maintained with molecule test harnesses:

```
1 roles/
2 apt_mirror/           # APT mirror configuration
3 beagleboard/          # BeagleBoard device provisioning
4 common/               # Shared base configuration (bootstrap.sh, common utils)
5 container_registry/   # Docker/container registry setup
6 ctfd/                 # CTFd platform deployment
7 desktop/              # Desktop environment provisioning
8 docker/               # Docker engine installation
9 documentation/        # Documentation generation roles
10 dhcp/                 # DHCP server configuration (dhcpd.conf)
11 dns/                  # BIND/named DNS server
12 golang/               # Go language toolchain
13 jetson-nano/          # NVIDIA Jetson Nano provisioning
14 k3s_agent/            # k3s worker node
15 k3s_server/           # k3s control plane server
16 k8s/                  # Kubernetes generic setup
17 latex/                # LaTeX environment
18 logging/              # Centralized logging
19 music/                # Music service configuration
20 nfs/                  # NFS server/client
21 nix/                   # Nix package manager
22 openbsd/              # OpenBSD host bootstrap (blowfish target)
23   tasks/main.yml      # Includes files.yml, timezone setup
```

```

24     files/                # bootstrap.sh, login.conf, hosts entries
25 paloalto/                # Palo Alto firewall config
26 prereq/                  # Prerequisite packages
27 pypi_internal/           # Internal PyPI repository setup
28     tasks/main.yml        # Symlink pypi_dir -> html_dir/pypi
29     vars/main.yml         # html_dir=/var/www/html, pypi_dir=/mnt/storage1/LAB/pypi
30 python/                  # Python environment
31 raspberrypi/             # Raspberry Pi provisioning
32 samba/                   # Samba file sharing
33 security/                # Hardening and security baselines
34 shell/                   # Shell configuration
35 ssh/                     # SSH server hardening
36 tls/                     # TLS certificate management
37 chonk/                   # chonk-specific configuration
38 cluster/                 # Cluster-wide settings
39

```

Roles currently **missing molecule tests**: edge, mail, odroid, website (incomplete roles). These have tasks/main.yml but no test harness.

14.18 Workspace Structure

The full project layout:

```

1 /workspace/lab-franklin/
2 Makefile.am                # Top-level autotools target (Python, Docker, dev)
3 configure.ac               # Autotools config (autoconf)
4 bootstrap.sh               # Cross-platform bootstrapper
5 network_update.sh          # One-shot maintenance script (hardened 2026-08-02)
6 ansible/                   # ANSIBLE_HOME root
7     playbook.yml           # Main playbook
8     hosts                  # Inventory file
9     collections/           # Collections path
10         ansible_collections/ # Galaxy namespace
11             lab/franklin/    # The lab/franklin collection
12                 galaxy.yml
13                 roles/       # All role modules (37 total)
14                 docs/latex/  # LaTeX docs build (autotools)
15 container/                 # Docker/Podman configs
16 terraform/                 # Terraform infrastructure definitions
17 bin/                       # Shared utility scripts (common.sh)
18 docs/manual/               # Lab manual documentation (this document)
19

```

14.19 Build System (Autotools)

The project uses GNU Autotools for the root-level build:

- **configure.ac**: Defines package (lab-franklin v0.2), libtool support, Python ≥ 3.9 requirement, podman/ansible/gpg checks, git version tagging
- **Makefile.am**: Top-level targets include test (ansible-lint + molecule), security (venv bootstrap), dev (Python venv in BUILDDIR)
- **bootstrap.sh**: Cross-platform bootstrapper that detects OS (Debian/RedHat/OpenBSD/macOS/Linux) and installs platform-specific packages, then runs aclocal $\rightarrow$ autoreconf $\rightarrow$ automake $\rightarrow$ configure
- **docs/manual/Makefile.am**: Standalone autotools stub for LaTeX docs build (clean target removes *.aux files)

The `network_update.sh` script was hardened on 2026-08-02 with: `set -euo pipefail`, auto-resolved paths, pinned K3s version validation, `ANSIBLE_HOMEEenvironmentchecks,removeddangerousclush/apt-getblastradius, anddeadcodere moval.Itrunsthemainplaybookatansible/playbook.ymlasitsprimaryaction.`

Testing from Stargate

14.20 Molecule Test Harnesses on stargate.lab.bitsmasher.net

The stargate host (10.10.16.66, Debian 12 bookworm) serves as the primary testing platform for the lab/franklin Ansible collection. Testing is run from `/mnt/clusterfs2/workspace/lab-franklin/ansible/collections`

14.20.1 Test Environment

Molecule tests on stargate use:

- **Driver:** podman (containers, not Docker – avoids daemon dependency)
- **Base image:** Ubuntu 20.04 for most roles, with role-specific variations
- **Execution user:** openclaw user (has sudo -n, no password required)
- **Python:** ≥ 3.9 (required by both ansible and molecule)

14.20.2 Test Execution Flow

For a given role, the molecule test cycle is:

1. **dependency:** Install collection dependencies from `galaxy.yml`
2. **syntax:** Validate playbook YAML syntax
3. **create:** Launch podman container (e.g., "server1") as the test target host
4. **prepare:** Verify role prerequisites within the container
5. **converge:** Run `ansible-playbook` against the container – this is where the role's tasks are executed
6. **test/verify:** Optional `pytest` sequence runs inside the container to validate convergence

14.20.3 Example: DNS Role Testing

The DNS role molecule test confirmed:

- Podman creates an Ubuntu 20.04 container (hostname "server1")
- Bind9 packages install correctly
- `/etc/bind` directory is created with `named.conf.options` written
- Molecule platform name was changed from "molecule-ubuntu" to "server1" to match the role's hostname gate
- Group assignment updated to "dns_servers" for proper role targeting

14.20.4 Known Limitations

Some roles cannot be fully tested on molecule containers:

- **nfs**: Hostname gates ('snowy' in `inventory_hostname`) and hardcoded mount paths depend on real NFS servers
- **openbsd**: The role targets OpenBSD hosts specifically; molecule tests run on Ubuntu containers by default
- **jetson-nano**: Hardware-specific (NVIDIA Jetson); molecule tests only verify file placement, not actual provisioning

14.20.5 Running Tests Manually

To test a specific role from the stargate host:

```
1 cd /mnt/clusterfs2/workspace/lab-franklin/ansible/collections/ansible_collections/
  lab/franklin/roles/<role>/molecule
2 molecule converge --scenario-name default
3
```

To verify convergence results:

```
1 molecule test --scenario-name default
2
```

If the podman driver is unavailable, fall back to docker:

```
1 molecule converge --driver-name docker
2
```

14.20.6 Test Results Dashboard

Testing output logs are captured in the molecule scenario directory under `logs/`. The full lab-franklin test suite (all roles with molecule tests) is run via the top-level Makefile.am `test` target:

```
1 make test
2 # Runs ansible-lint --all-roles and molecule converge per role
3
```

14.20.7 pytest-ansible Transition Plan

As molecule is being superseded, new tests should use pytest-ansible where possible:

- Migrate the test directory structure from `molecule/` to a `tests/` directory with pytest configuration
- Use `ansible-test` for collection sanity tests (built into `ansible-core`)
- Keep molecule only where full environment isolation is needed (e.g., DNS zone file validation, NTP time sync testing)

14.20.8 Pending Tasks and Todos

- **TODO: Re-establish internal PyPI server on stargate.** The `pypi_internal` role is ready (role exists, molecule test added Aug 2026) but the actual PyPI service needs to be provisioned on the storage host. Requires: NFS mount of `/mnt/storage1` available on stargate, web server (nginx/apache) installed and configured, pip packages populated into `/mnt/storage1/LAB/pypi` with PEP 503 structure. Run: `ansible-playbook ... -l storage --tags pypi_internal` once the infrastructure is in place.

14.21 Ansible-Linting Infrastructure

All ansible roles in the lab collection should be linted on a regular basis to maintain configuration quality and catch issues early. The lab uses three tools: `ansible-lint`, `yamllint`, and `ansible-test` `sanity`.

14.21.1 Tools and Configuration

Each linter is configured in the `test/` directory:

- `.ansible-lint.yml` — ansible-lint configuration with warnings for deprecated modules and experimental rules disabled
- `.yamllint / .yamllintrc` — YAML linting extends default rules with lab-specific indentation and spacing constraints

14.21.2 Coverage Status (44 Roles)

Based on the most recent audit of `ansible/collections/ansible_collections/lab/franklin/roles/`:

Complete (30 roles) apt-mirror, beagleboard, chonk, cluster, common, container-registry, ctf, desktop, dhcp, dns, docker, documentation, golang, k3s-agent, k3s-server, kerberos, ldap, logging, minecraft, music, nfs, nix, ntp, openbsd, paloalto, prereq, pypi-internal, python, raspberrypi, ssh

Minimal/Functional (9 roles) media, samba, shell, tls, website, apt-mirror-stub, jetson-nano, k8s (partial), dns-stub

Stubs (4 roles) edge, extensions, latex, odroid

Roles with the smallest role definitions tend to have the fewest linting issues. The most problematic roles are usually those that include dynamic template generation or custom module invocation.

14.21.3 Proposed Linting Pipeline

A new `test/lint_all.sh` script should be created to run all three linters in sequence:

```
# Example lint_all.sh structure:
#!/bin/bash
set -euo pipefail

cd "$(dirname "$0")/.."

echo "=== ansible-lint ==="
ansible-lint roles/ 2>&1 || echo "ansible-lint warnings found"

echo "=== yamllint ==="
yamllint -c test/.yamllint roles/ 2>&1 || echo "yamllint issues found"

echo "=== ansible-test sanity ==="
ansible-test sanity --docker default -v 2>&1 || echo "sanity check failures"
```

The linting pipeline should be integrated into:

- **GitHub Actions:** Run on PR and nightly builds (similar to existing bandit/trivy/tfsec workflows)
- **Hourly cron job on stargate:** Optional for CI-style validation without external triggers

14.21.4 Linting Exceptions by Role

Some roles require special handling during linting:

- **tls:** Includes shell commands with `kss -kfe` which may trigger warning 403 (deprecated module) — explicitly skip in config
- **samba:** Uses custom `smb.conf.j2` templates that require validation via `testparm` post-converge
- **k8s:** Contains static manifests rather than role tasks — should be moved to a shared files directory or documented as a reference role

Terraform Infrastructure Management

Terraform configurations in this lab manage cloud infrastructure across two providers: DigitalOcean and Google Cloud Platform. All configurations are version-controlled and include automated security scanning via GitHub Actions.

14.22 DigitalOcean Infrastructure

The `terraform/digital_ocean/` directory manages the external-facing web presence for bitasmasher.net.

14.22.1 Provisioned Resources

- **Droplet:** 512MB Debian 12 in lon1 region (www.bitasmasher.net)
- **Domain:** bitasmasher.net with full DNS management
- **DNS Records:** A, MX, TXT (SPF/DMARC/DKIM), NS, and ACME challenge records

14.22.2 Key DNS Records

Name	Type	Purpose
www.bitasmasher.net	A	Web server host (178.62.60.55)
@	MX	mail.protonmail.ch 10 (ProtonMail routing)
@	TXT	SPF and DMARC policies
protonmail._domainkey	CNAME	DKIM email verification
_acme-challenge.*.lab	TXT	Let's Encrypt certificate challenges

14.22.3 Configuration Notes

The droplet includes:

- Automated backups enabled (immutable against accidental deletion)
- SSH key injection via terraform data source
- No external monitoring (consider enabling DO agent for uptime tracking)

Provider authentication uses a DigitalOcean API token managed through pass/direnv integration. The token is passed at runtime, never committed to the repository.

14.23 Google Cloud Platform Infrastructure

The `terraform/google/` directory manages GCP resources for Stash House free-tier operations.

14.23.1 Resources Managed

- **VPC Network:** `stash-house-vpc` dedicated network
- **Compute Instance:** `e2-micro` Debian 12 (Always Free Tier eligible) in `us-central1-a`
- **Firewall Rule:** `allow-ssh` opening port 22 to the VPC
- **Billing Budget Alert:** Tripwire alert at \$0.01 threshold for free-tier monitoring

The GCP configuration is intentionally minimal, designed around Google's Always Free Tier limits to maintain zero-cost infrastructure.

14.24 Security Scanning

All Terraform configurations are automatically scanned by two GitHub Actions workflows:

- `tfsec-sarif.yml` — IaC security scanning (generates SARIF reports)
- `tfsec-comment.yml` — Posts security findings as PR comments

Additionally, the DigitalOcean directory includes existing `README.md` with `doctl` and `pass` integration instructions for manual operations.

14.25 Testing

Terraform configurations are validated through:

- Go-based integration tests (`terraform/test/main_test.go`)
- Plan dry-run validation via GitHub Actions (`markdown.yml` workflow)
- Manual state reconciliation checks (compare terraform state against live infrastructure)

State drift detection is recommended as a regular maintenance task — compare `terraform state list` output against actual cloud resource inventories quarterly.